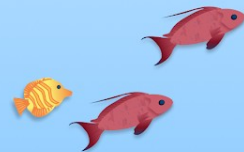


Wakacyjne Wyzwanie

Z  KN Solvro

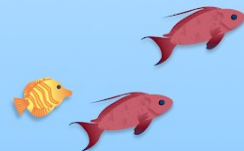


Wykład 4

Wprowadzenie do konteneryzacji

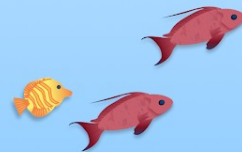


Plan na dziś



- Czym jest konteneryzacja
 - Konteneryzacja vs wirtualizacja
 - Koncepty w konteneryzacji
- „Silniki” kontenerów
- Jak skonfigurować jednokontenerowy serwis
- Jak postawić wiele serwisów HTTP na jednym porcie
- Kwestie bezpieczeństwa
 - Czyli czego unikać przy korzystaniu z kontenerów



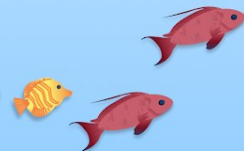


Rozdział 1

Czym jest konteneryzacja?



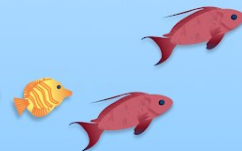
Gdy wirtualizacja jest zbyt ciężka...



- Co jeżeli chcemy zapewnić, by każdy serwis był od siebie odpowiednio odizolowany?
- Co jeśli chcielibyśmy niektóre serwisy uruchamiać w bardziej minimalnych środowiskach?
- Co gdy różne serwisy wymagają różnych środowisk? (np. dystrybucji)
 - ...ale jednocześnie nie potrzebują wyspecjalizowanego kernela
- Co jeśli chcielibyśmy prosto i szybko stawiać więcej instancji serwisów?
- Stawianie 1 VMki per serwis nie wydaje się być dobrym pomysłem...



Gdy wirtualizacja jest zbyt ciężka...

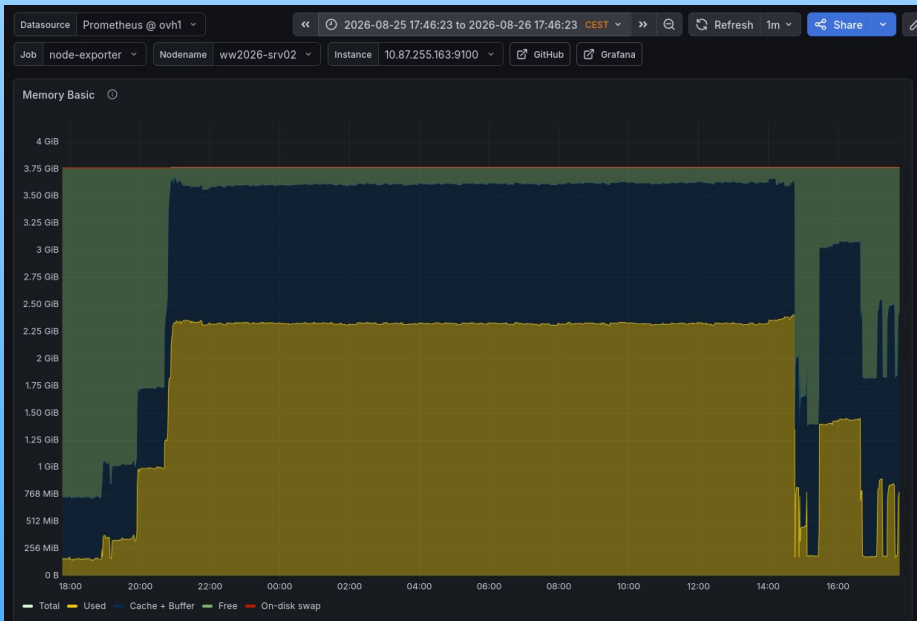
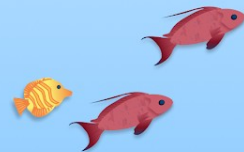


- Przypomnienie kwestii przydzielania zasobów:

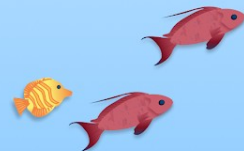
- CPU – bez problemu
- Dysk – wolne miejsce zwalniane co kilka dni
- RAM – yyyyyyy
 - ▶ Linux o RAMie: niewykorzystany ram to zmarnowany
 - ▶ RAM – wolny RAM jest wykorzystywany jako cache
 - ▶ W teorii można by skomunikować VMke z wirtualizatorem, by samo zwalniało nieużywany ram



Przykład z udostępnionych VMek



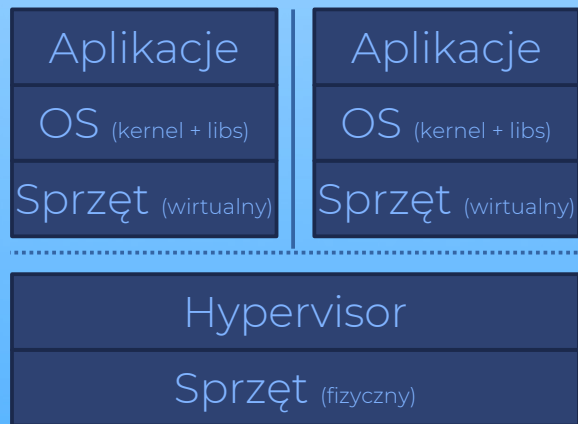
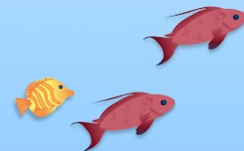
Czym jest konteneryzacja?



- Podobnie do wirtualizacji – chcemy odizolować kontenery od siebie
- Ale zamiast tworzyć wirtualny sprzęt, na którym uruchamiamy system – korzystamy z funkcji systemu hosta, pozwalających na wydzielanie osobnych środowisk
 - System przestrzeni – namespace typu mount, UTS, IPC, network, PID, cgroup, user, time
 - + chroot

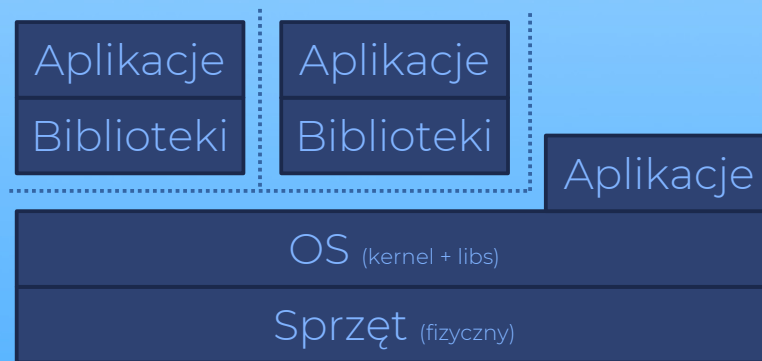


Konteneryzacja vs wirtualizacja



Wirtualizacja

Izolacja poprzez wirtualizację sprzętu i wydzielenie pamięci



Konteneryzacja

Izolacja poprzez funkcje systemu operacyjnego

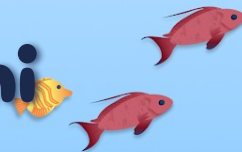


Jak działają systemy przestrzeni w Linuxie?

- Różne typy przestrzeni grupują różne zasoby
 - mount namespace ogranicza zmiany w zamontowanych systemach plików
 - PID namespace ukrywa pozostałe procesy (i zmienia identyfikatory procesów)
 - network namespace tworzy wirtualnego hosta sieciowego z osobnymi urządzeniami
 - user namespace zmienia identyfikatory użytkowników
 - Pozornie „zwykli” użytkownicy wewnątrz są mapowani do nieistniejących użytkowników poza
 - Może ograniczyć uprawnienia UID 0 / CAP_DAC_OVERRIDE
- **chroot zmienia katalog /**
- **cgroup używane do ograniczenia zużycia CPU lub RAM**



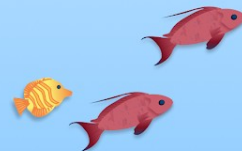
Różnice między kontenerami a VMkami



- **Kontenery są bardziej zintegrowane z OS**
 - System operacyjny widzi co się dzieje w kontenerach
- **Kontenerom nie przydzielamy zasobów wirtualnych**
 - Nie musimy podawać z góry ile CPU/RAM/dysku potrzeba
 - Możemy ograniczyć zużycie poprzez cgroup
 - Nie przydzielamy RAMu = linux nie marnuje go na potencjalnie bezużyteczny cache systemu plików
- **Kontenery są z reguły mniejsze**
 - Nie potrzeba kernela, bootloadera, systemu init, zdalnego dostępu...
 - Najprostszy kontener może dosłownie zawierać tylko 1 plik – aplikację
- **Kontenery z reguły są nastawione na prostą replikację**
 - Aplikacja nie wyrabia? Odpal kolejny kontener z tego samego obrazu!
- **Kontenery powinny mieć wyznaczone miejsca na dane**
 - Zmiany w pozostałych miejscach mogą być cofane lub blokowane
 - Zakłada się, że kontener można w każdym momencie usunąć i stworzyć od nowa



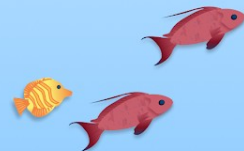
Koncepty konteneryzacji



- **Obraz** – Bazowy system plików dla jakiejś aplikacji
 - Składa się z warstw
 - Można z niego tworzyć kontenery lub kolejne obrazy
- **Kontener** – Żywa instancja obrazu
 - Obraz złożony w całość
 - wraz z dedykowaną warstwą na zmiany lokalne
 - oraz z dodatkowymi zamontowanymi katalogami/wolumenami
 - Utworzone wszystkie wymagane przestrzenie
 - Uruchomiona aplikacja w tych przestrzeniach
 - Podpięte do sieci, wolumenów, opublikowane porty
 - Monitorowanie stanu aplikacji (jeżeli skonfigurowano)



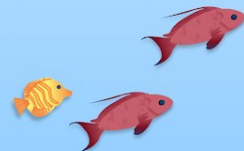
Koncepty konteneryzacji



- **Dockerfile/Containerfile** – skrypt do tworzenia obrazów
 - Opisuje kroki wymagane do utworzenia obrazu
 - Każdy krok to osobna warstwa w gotowym obrazie
 - Kroki typu „skopiuj plik do obrazu”, „uruchom komendę w obrazie”, „ustaw zmienną środowiskową”, itp.
 - Może bazować na istniejących obrazach
- **Wolumen** – katalog przeznaczony do montowania w kontenerach
 - Może służyć do trwałego przechowywania danych
 - ...lub do współdzielenia danych między kontenerami
 - ...lub do komunikacji między kontenerami i/lub hostem
 - unix sockets ❤️

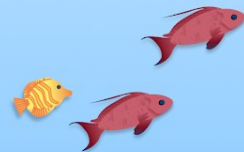


Koncepty konteneryzacji



- Sieć – wirtualna sieć zarządzana przez silnik, do komunikacji między kontenerami
 - Sieć domyślna + sieci własne
 - Zazwyczaj tworzone poprzez interfejsy bridge z NAT
 - W przypadku klastrów kontenerowych – tzw. sieć overlay
 - Możliwość odcięcia kontenera od sieci zewnętrznej
 - Możliwość „publikowania” portów
 - czyli silnik dodaje reguły DNAT (port forward) do firewalla
- **Compose – plik deklarujący zestaw kontenerów, wolumenów i sieci**
 - Wykorzystywany w aplikacjach złożonych z wielu serwisów
 - Zarówno mikroservisy, jak i monolit z bazami danych
 - Może deklarować zależności między kontenerami (uruchom X gdy Y się ustabilizuje) i kontenery jednorazowe (do uruchomienia instalacji/migracji/aktualizacji)
 - O tym więcej w następnym tygodniu



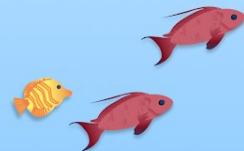


Rozdział 2

Silniki kontenerów



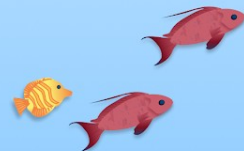
Czym jest ten silnik?



- Program, który zarządza obiektami kontenerów
 - Pobiera, tworzy i zarządza obrazami
 - Zarządza sieciami, tworzy odpowiednie reguły w firewallu
 - Tworzy i zarządza wolumenami
 - Uruchamia kontenery według specyfikacji
 - Może decydować, na którym komputerze uruchomić kontener
 - ▶ Może przenosić kontenery między komputerami



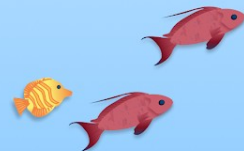
Jakie mamy silniki?



- **Docker**
 - Najpopularniejszy
 - Dobry do uruchamiania na pojedynczych komputerach
 - Ma również tryb działania w klastrze (Docker swarm)
 - **Osobista opinia: brakuje mu czasami prostych, ale potrzebnych funkcji**
 - np. eksportowanie/importowanie wolumenów (do tworzenia kopii zapasowych)
 - **Działa jako serwis w tle**
 - Komenda docker po prostu wysyła do tego serwisu żądania
 - **Serwis działa jako root**
 - **Dostęp do serwisu docker daje możliwości podobne do sudo (ale bez hasła!)**
 - Więcej w rozdziale 5
 - Korzysta z iptables
 - Czyli trudno jest ograniczyć we własny sposób dostęp kontenerów do sieci (lub dostęp do kontenerów z sieci)



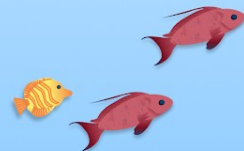
Jakie mamy silniki?



- **Podman**
 - Kompatybilny z dockerem, ale ma więcej funkcji
 - I dlatego jest to mój ulubiony silnik kontenerów
 - Wyczerpująca dokumentacja w man
 - Przeznaczony do działania na pojedynczym komputerze
 - Działa bez serwisu w tle
 - Uruchamia kontenery bezpośrednio
 - Serwis w tle wspierany jako kompatybilność z dockerem, lub jako metoda zarządzania zdalnego
 - Znakomita integracja z systemd
 - Możliwość deklaracji obiektów kontenerowych jako jednostki (serwisy) systemd
 - Wspiera uruchamianie z ograniczonymi uprawnieniami
 - Zwykli użytkownicy (bez root/sudo) mogą bezpiecznie uruchamiać własne kontenery
 - Korzysta z nftables
 - Reguły tworzone przez podman nie konfliktują z własnymi regułami {ip,nf}tables



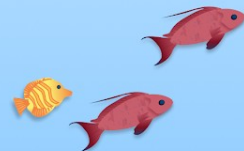
Jakie mamy silniki?



- **Kubernetes**
 - Przeznaczony do działania w klastrze
 - Używa własnego formatu specyfikacji (zestawu) obiektów – kubernetes manifest
- **systemd-nspawn**
 - Kolejny sposób, tym razem przez systemd
 - Przeznaczony bardziej do tworzenia kontenerów całych systemów/dystrybucji, niż pojedynczych aplikacji
 - Uruchamia systemd wewnątrz kontenera
- **LXC (Linux Containers)**
 - Jeszcze inny sposób na kontenery...

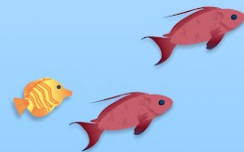


2 których silników korzystać?



- Kurs będzie skupiał się na kontenerach w stylu dockerowym
 - Czyli z wykorzystaniem docker lub podman
- Pozostałe silniki zostały zaprezentowane jako ciekawostka
 - Kubernetes był tematem Wakacyjnego Wyzwania DevOps 2025



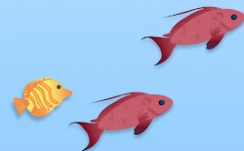


Rozdział 3

Proste konfiguracje kontenerów



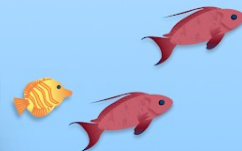
Tworzenie kontenera



- **Komenda `docker run [flagi] <nazwa obrazu> [argumenty linii poleceń]`**
 - Gotowe obrazy można znaleźć na stronie hub.docker.com
 - ▶ Docker domyślnie szuka obrazów tutaj, podman często wymaga pełnych nazw
 - ▶ `<repozytorium>/<nazwa obrazu>:<tag>`
 - np. ubuntu => docker.io/library/ubuntu:latest
 - Argumenty linii poleceń zależą od obrazu
 - Ważne flagi na początek:
 - ▶ `-i, -t` => Tryb interaktywny, utwórz wirtualny terminal
 - ▶ `-d` => Uruchom w tle
 - ▶ `-h <nazwa>` => ustaw nazwę hosta wewnątrz kontenera
 - ▶ `--name <nazwa>` => ustaw zewn. nazwę kontenera (dla pozostałych komend)
 - ▶ `--rm` => usuń kontener po zatrzymaniu



Uruchamianie pierwszego kontenera



```
~ : zsh — Konsola

Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc

Nowa karta  Podziel widok  Zezwól na śledzenie myszy  Skopiuj  Wklej  Poszukaj

mini_bomba@miniarchiumIV ~$ podman run docker.io/library/hello-world:latest
Trying to pull docker.io/library/hello-world:latest...
Getting image source signatures
Copying blob 4f55086f7dd0 done |
Copying config e2ac70e731 done |
Writing manifest to image destination

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

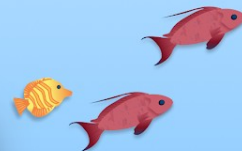
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

mini_bomba@miniarchiumIV ~$ podman container ls -a | grep hello
837b09d3ba4e  docker.io/library/hello-world:latest  /hello  4 seconds ago  Exited (0) 5 seconds ago
               clever_goldstine
mini_bomba@miniarchiumIV ~$ podman rm clever_goldstine
clever_goldstine
mini_bomba@miniarchiumIV ~$
```



Kontenery interaktywne



```
~: zsh — Konsola
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc
Nowa karta  Podziel widok  Zezwól na śledzenie myszy  Skopiuj  Wklej  Poszukaj

mini_bomba@miniarchiumIV ~$ podman run -it debian-test --name debian-test --pull newer docker.io/library/debian:latest
root@debian-test:/# apt update && apt install python3
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9673 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [4412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [250 kB]
Fetched 10.2 MB in 1s (7424 kB/s)
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  libssl3t64  openssl-provider-legacy

Installing:
  python3

Installing dependencies:
  ca-certificates  libgpm2  libpython3.13-minimal  media-types  python3-minimal  readline-common
  libexpat1        libncursesw6  libpython3.13-stdlib  netbase      python3.13
  libffi8          libpython3-stdlib  libreadline8t64      openssl      python3.13-minimal

Suggested packages:
  gpm  python3-doc  python3-tk  python3-venv  python3.13-doc  binutils  binfmt-support  readline-doc

Summary:
  Upgrading: 2, Installing: 17, Removing: 0, Not Upgrading: 0
  Download size: 10.9 MB
  Space needed: 27.4 MB / 184 GB available

Continue? [Y/n]
Get:1 http://deb.debian.org/debian-security trixie-security/main amd64 libexpat1 amd64 2.8.3-1~deb13u1 [123 kB]
Get:2 http://deb.debian.org/debian-security trixie-security/main amd64 openssl-provider-legacy amd64 3.5.7-1~deb13u2 [316 kB]
Get:3 http://deb.debian.org/debian-security trixie-security/main amd64 libssl3t64 amd64 3.5.7-1~deb13u2 [2454 kB]
Get:4 http://deb.debian.org/debian trixie-updates/main amd64 libpython3.13-minimal amd64 3.13.5-2+deb13u4 [863 kB]
```



Restartowanie trwałych kontenerów

```
~ : zsh — Konsola
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc
Nowa karta  Podziel widok  Zezwól na śledzenie myszy  Skopiuj  Wklej  Poszukaj

mini_bomba@miniarchiumIV ➤ podman container ls -a | grep debian
a8a7641cf80e  docker.io/library/debian:latest  bash  2 minutes ago  Exited (137) 9 seconds ago
               debian-test

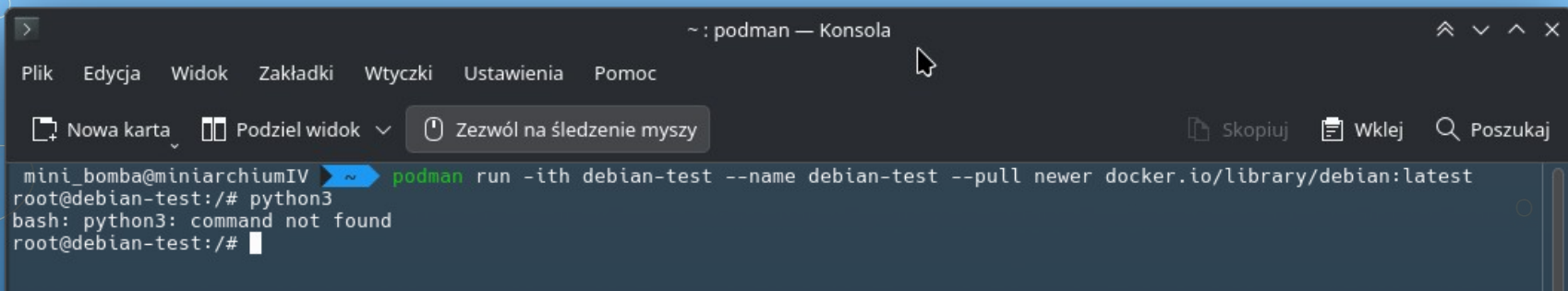
mini_bomba@miniarchiumIV ➤ podman start debian-test
debian-test

mini_bomba@miniarchiumIV ➤ podman attach debian-test
root@debian-test:/#
root@debian-test:/# python3
Python 3.13.5 (main, Jul 15 2026, 20:25:40) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("elo zelo!")
elo zelo!
>>>
root@debian-test:/#
exit

mini_bomba@miniarchiumIV ➤ podman rm debian-test
debian-test
mini_bomba@miniarchiumIV ➤
```



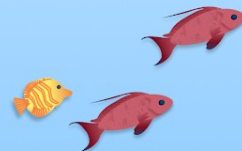
Modyfikacje w kontenerze nie wpływają na obraz



```
mini_bomba@miniarchiumIV ~$ podman run -ith debian-test --name debian-test --pull newer docker.io/library/debian:latest
root@debian-test:/# python3
bash: python3: command not found
root@debian-test:/#
```



Więcej ważnych flag



- -p <port hosta>:<port kontenera> => wystaw serwis z portu kontenera na port hosta
- -v <ścieżka na hoście>:<ścieżka w kontenerze>:<flagi> => zamontuj katalog z hosta do kontenera
- -v <nazwa wolumenu>:<ścieżka w kontenerze>:<flagi> => zamontuj wolumen w kontenerze



Wystawianie usług

```
tmp : zsh — Konsola
Plik Edycja Widok Zakładki Wtyczki Ustawienia Pomoc
Nowa karta Podziel widok Zezwól na śledzenie myszy Skopiuj Wklej Poszukaj

mini_bomba@miniarchiumIV ➤ /tmp cat container-files/index.html
File: container-files/index.html
1 elo żelo!!!!

mini_bomba@miniarchiumIV ➤ /tmp podman run -dh nginx --name nginx-test --rm -v ./container-files:/usr/share/nginx/html:ro --p
ull newer -p 8080:80 docker.io/library/nginx
703193a2a7e95cb847dbf83ea3f6554a6b0a0373f0b1d361499d259f779366d8
mini_bomba@miniarchiumIV ➤ /tmp curl -v http://localhost:8080
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8080...
* Established connection to localhost (::1 port 8080) from ::1 port 37714
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.21.0
> Accept: */*
*
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.31.4
< Date: Sat, 29 Aug 2026 14:03:20 GMT
< Content-Type: text/html
< Content-Length: 15
< Last-Modified: Sat, 29 Aug 2026 13:58:38 GMT
< Connection: keep-alive
< ETag: "6a92e58e-f"
< Accept-Ranges: bytes
*
elo żelo!!!!
* Connection #0 to host localhost:8080 left intact
mini_bomba@miniarchiumIV ➤ /tmp podman stop nginx-test
nginx-test
mini_bomba@miniarchiumIV ➤ /tmp
```



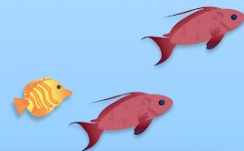
Wolumeny

```
tmp: zsh — Konsola
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc
Nowa karta  Podziel widok  Zezwól na śledzenie myszy  Skopiuj  Wklej  Poszukaj

mini_bomba@miniarchiumIV > /tmp podman volume create test-vol
test-vol
mini_bomba@miniarchiumIV > /tmp podman run -ith debian-test --name debian-test --pull newer -v test-vol:/host docker.io/library/debian:latest
root@debian-test:/# ls /host
root@debian-test:/# echo elozelo > /host/elozelo.txt
root@debian-test:/# cat /host/elozelo.txt
elozelo
root@debian-test:/# echo 1234 > /pozawolumenem.txt
root@debian-test:/# cat /pozawolumenem.txt
1234
root@debian-test:/#
exit
mini_bomba@miniarchiumIV > /tmp podman rm debian-test
debian-test
mini_bomba@miniarchiumIV > /tmp podman run -ith debian-test --name debian-test --pull newer -v test-vol:/host docker.io/library/debian:latest
root@debian-test:/# ls /host
elozelo.txt
root@debian-test:/# cat /host/elozelo.txt
elozelo
root@debian-test:/# cat /pozawolumenem.txt
cat: /pozawolumenem.txt: No such file or directory
root@debian-test:/#
exit
* mini_bomba@miniarchiumIV > /tmp podman rm debian-test
debian-test
mini_bomba@miniarchiumIV > /tmp podman volume rm test-vol
test-vol
mini_bomba@miniarchiumIV > /tmp
```



Własne obrazy



```
test : vim — Konsola
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia
Nowa karta  Podziel widok
1 FROM docker.io/library/archlinux:latest
2 COPY ./entrypoint.sh /entrypoint.sh
3 ENV EL0="żelo"
4 ENTRYPOINT ["/entrypoint.sh"]
/tmp/test/Dockerfile
1 #!/bin/bash
2 echo "elo $EL0!"
N entrypoint.sh  sh  1:1
```

```
test : zsh — Konsola
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc
Nowa karta  Podziel widok  Zezwól na śledzenie myszy  Skopiuj
mini_bomba@miniarchiumIV /tmp/test podman build -t elo-zelo .
STEP 1/4: FROM docker.io/library/archlinux:latest
STEP 2/4: COPY ./entrypoint.sh /entrypoint.sh
--> 3ef1f5b4396b
STEP 3/4: ENV EL0="żelo"
--> ec97aec4fbba
STEP 4/4: ENTRYPOINT ["/entrypoint.sh"]
COMMIT elo-zelo
--> 214189b1a08d
Successfully tagged localhost/elo-zelo:latest
214189b1a08db7d67311d83a432508f5a70659e7cf414d3ac46b0f7088198de7
mini_bomba@miniarchiumIV /tmp/test podman run -ith test --rm --name test
elo-zelo
elo żelo!
mini_bomba@miniarchiumIV /tmp/test podman run -ith test --rm --name test
-e "EL0=wakacyjne wyzwanie devops 2026" elo-zelo
elo wakacyjne wyzwanie devops 2026!
mini_bomba@miniarchiumIV /tmp/test
```

Dokumentacja Dockerfile:

docs.docker.com/reference/dockerfile

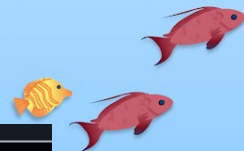


Szablon © Basia Czastkiewicz 2026
Treści © Konrad Szczepaniak 2026
CC BY-NC 4.0



KN Solvro

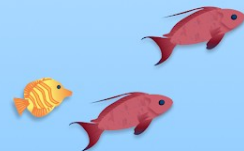
Bardziej skomplikowane dockerfile



```
1  # syntax=docker/dockerfile:1.23
2  FROM docker.io/library/rust:alpine AS builder
3  # add only what's necessary
4  COPY --parents src/ Cargo.lock Cargo.toml /source/
5  WORKDIR /source
6  # compile
7  RUN --mount=type=cache,target=/usr/local/cargo/registry,id=alpine_cargo_dir \
8      --mount=type=cache,target=/source/target,id=ip_checker_target \
9      --mount=type=bind,target=/context \
10     touch src/build_script.rs && \
11     GIT_DIR=/context/.git cargo build --release --locked && \
12     cp /source/target/release/ip-checker /
13
14  # prod image
15  FROM scratch
16  USER 1000:1000
17  COPY --from=builder /ip-checker /ip-checker
18  ENTRYPOINT ["/ip-checker"]
19  HEALTHCHECK --start-period=1s --interval=15s --timeout=1s CMD ["/ip-checker", "health"]
```



Plik .dockerignore



- Lista plików/katalogów, które powinny zostać ukryte podczas tworzenia obrazu
- Zapobiega przypadkowemu dodaniu plików niebędących kodem źródłowym do kontenera
 - Powinien zawierać listę katalogów zawierających automatycznie pobrane biblioteki, skompilowane wersje programu, lub pliki zawierające poświadczenia

devops-ip-checker / .dockerignore



mini-bomba fix: un-dockerignore .git ✓

Code

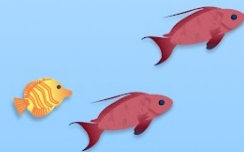
Blame

1 lines (1 loc) · 8 Bytes ·

1

/target



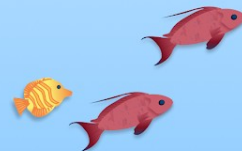


Rozdział 4

Reverse proxy



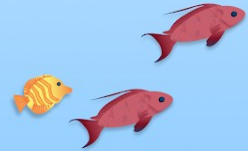
Problem z portami



- Co jeżeli chcemy wystawić dwie usługi po HTTP?
 - HTTP nie zakłada możliwości zmiany domyślnego portu według domeny
 - HTTPS już to wspiera, ale wymagane jest wsparcie przeglądarki
 - Nie chcemy, by użytkownicy musieli ręcznie wpisywać port



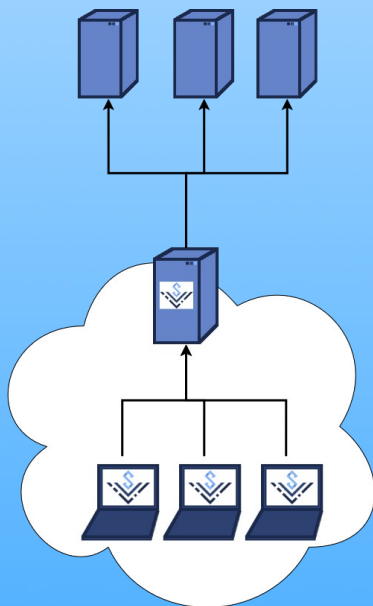
Reverse proxy



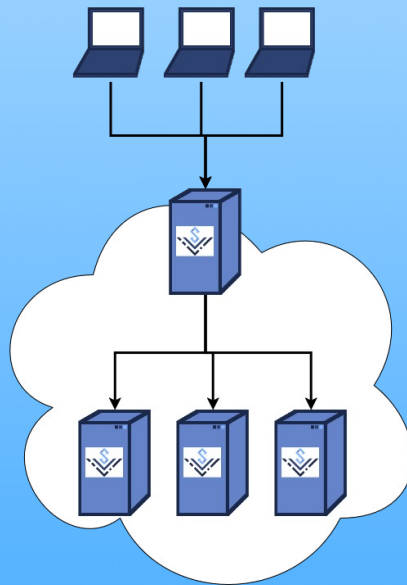
- Zadaniem reverse proxy jest przyjmowanie żądań z zewnątrz i kierowanie ich do odpowiedniego serwisu wewnątrz
 - Zazwyczaj po nazwie domeny, czasami po ścieżce
- Często „kończy” ono połączenie szyfrowane HTTPS i używa dalej HTTP
 - ... albo korzysta z certyfikatów self-signed, nieuznawanych przez przeglądarki
 - „TLS termination”
- Może pełnić dodatkowe funkcje, np. WAF lub cache



Dlaczego „reverse” proxy?



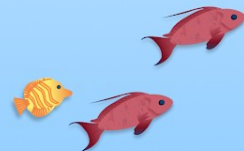
Forward proxy



Reverse proxy



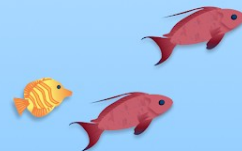
Oprogramowanie reverse proxy



- **Apache httpd**
 - Stare, znane, większość PHPiarzy na tym bazuje
 - W mojej opinii lekko przestarzałe
 - Głównie serwer HTTP z funkcjami reverse proxy
- **nginx**
 - Też stare, ale moje ulubione
 - „HTTP web server, reverse proxy, content cache, load balancer, TCP/UDP proxy server, and mail proxy server”
- **HAProxy**
 - Reverse proxy i load balancer
 - Używane w faktycznych firmach, przy dużej skali



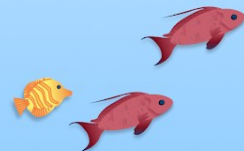
Opogramowanie reverse proxy



- **Caddy**
 - Nowszy serwer HTTP + reverse proxy
 - Wiele typów konfiguracji (w tym dynamiczna poprzez labelę kontenerów)
 - Automatyczne generowanie certyfikatów HTTPS
- **Traefik**
 - Reverse proxy/load balancer przeznaczony dla środowisk kontenerowych
 - Konfiguracja dynamiczna poprzez labelę kontenerów
 - Automatyczne generowanie certyfikatów HTTPS



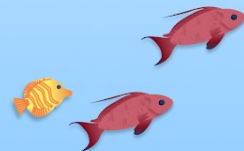
2 którego korzystać?



- **Osobiście korzystam z nginx**
 - Jest w miarę prosty
 - ...i dlatego na kursie również będzie nginx
- **W Solvro korzystamy z traefik**
 - Konfiguracja jest trochę bardziej skomplikowana
 - I trzeba o niej myśleć z wyprzedzeniem – etykiet nie da się modyfikować po utworzeniu kontenera
- **Caddy podobno też jest fajną opcją**



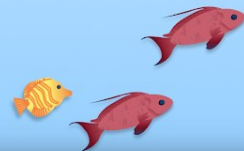
Stawianie nginxa



- nginx jest dostępny w repozytorium praktycznie każdego distro
- Dokumentacja konfiguracji: nginx.org/en/docs/
- Na debianie konfiguracje stron można dodawać do `/etc/nginx/sites-enabled`
 - Na innych dystrybucjach może być wymagana ręczna konfiguracja w `/etc/nginx/nginx.conf`



Prosty serwer HTTP

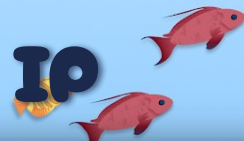


```
1 server {  
2     listen [::]:80 ipv6only=off;  
3     server_name "test.localhost";  
4     root /srv/http/test;  
5 }
```

```
(root) ovh1.minibomba.pro — Konsola  
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc  
Nowa karta  Podziel widok  Zezwól na śledzenie myszy  Skopiuj  Wklej  Poszukaj  
# root@ovh1 in /srv/http/test [17:26:04]  
$ cat index.html  
File: index.html  
1 <h2>elo żelo!!!!</h2>  
# root@ovh1 in /srv/http/test [17:26:06]  
$ curl -v http://test.localhost  
* Host test.localhost:80 was resolved.  
* IPv6: ::1  
* IPv4: 127.0.0.1  
* Trying [::1]:80...  
* Established connection to test.localhost (::1 port 80) from ::1 port 50412  
* using HTTP/1.x  
> GET / HTTP/1.1  
Host: test.localhost  
User-Agent: curl/8.21.0  
Accept: */*  
>  
* Request completely sent off  
< HTTP/1.1 200 OK  
< Server: nginx/1.30.4  
< Date: Sat, 29 Aug 2026 15:26:09 GMT  
< Content-Type: text/html; charset=utf-8  
< Content-Length: 23  
< Last-Modified: Sat, 29 Aug 2026 15:24:50 GMT  
< Connection: keep-alive  
< ETag: "6a92f9c2-17"  
< Accept-Ranges: bytes  
<  
<h2>elo żelo!!!!</h2>  
* Connection #0 to host test.localhost:80 left intact  
# root@ovh1 in /srv/http/test [17:26:18]  
$
```



Reverse proxy + przekazywanie adresu IP



```
1 server {
2     listen [::]:80;
3     server_name test2.localhost;
4
5     location / {
6         proxy_pass http://localhost:8080;
7
8         proxy_set_header X-Forwarded-Proto $scheme;
9         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
10        proxy_set_header X-Forwarded-Port $remote_port;
11        proxy_set_header X-Forwarded-Host $host;
12        proxy_set_header X-Real-IP $remote_addr;
13    }
14 }
```

```
(root) ovh1.minibomba.pro — Konsola
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc
Nowa karta  Podziel widok  Zezwól na śledzenie myszy

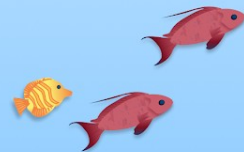
(root) ovh1.minibomba.pro x  tmp: zsh x

# root @ ovh1 in ~/devops-ip-checker on git:main o [17:41:25]
$ podman run -dh test --rm --name test --users=auto --pull newer -p 8080:8080 -e IP_CHECKER=ea6a:ad4a:d048::1/128"`, "forwarded_headers": ["X-Real-IP"]}` ip-checker 7ac8cc72491cedb86c2980b0c8db8be71e77e2dd3f4d32f842ee0643137e6742

# root @ ovh1 in ~/devops-ip-checker on git:main o [17:41:31]
$ curl -v http://test2.localhost
* Host test2.localhost:80 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:80...
* Established connection to test2.localhost (::1 port 80) from ::1 port 53318
* using HTTP/1.x
> GET / HTTP/1.1
> Host: test2.localhost
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.30.4
< Date: Sat, 29 Aug 2026 15:41:34 GMT
< Content-Type: text/plain; charset=utf-8
< Content-Length: 90
< Connection: keep-alive
<
Twoje IP: ::1
* Connection #0 to host test2.localhost:80 left intact
devops-ip-checker 0282ff1f (refs/heads/main) Sat, 29 Aug 2026 15:28:37 +0000#

# root @ ovh1 in ~/devops-ip-checker on git:main o [17:41:34]
$
```



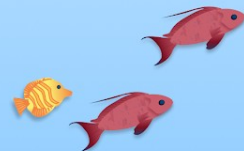


Rozdział 5

Bezpieczeństwo konteneryzacji



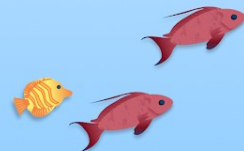
Niebezpieczne konfiguracje



- Należy mieć na uwadze uprawnienia dostępu do daemona docker/podman uruchomionego jako root!
 - /run/docker.sock, /run/podman.sock
 - Domyślnie ograniczone do grupy docker
 - Każdy kto ma dostęp do tego gniazda ma efektywnie uprawnienia root!
- **Root w kontenerze = root na hoście**
 - docker run -it -v /:/host docker.io/library/debian
 - ... wystarczy hasło roota w /host/etc/shadow i mamy roota poza kontenerem



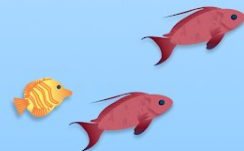
Niebezpieczne konfiguracje



- Montowanie `/run/docker.sock` do kontenerów pozwala im zarządzać kontenerami!
 - ... i tak samo pozwoli im również wprowadzić modyfikacje poza kontenerem!
- Kontenery mają domyślnie przycięte „capabilities”
 - Ogranicza to akcje, które można w nich podejmować, nawet jako root!
 - np. domyślnie kontener nie może zmieniać własnej konfiguracji sieci
 - Capabilities można przywracać flagą `--cap-add` lub `--privileged`
 - Może to umożliwić ucieczkę programu z kontenera!



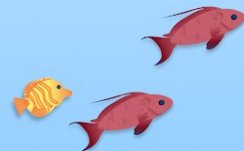
Utwardzanie kontenerów



- Jeżeli możliwe, uruchamiaj programy wewnątrz kontenera jako użytkownik inny niż UID 0 (root)
- Unikaj użycia flag `--privileged` i `--cap-add`
- Nie przypisuj żadnemu użytkownikowi grupy docker
 - Zamiast tego korzystaj z `sudo`
- Montuj w kontenerach tylko wymagane wolumeny/katalogi
 - W szczególności nie montuj całych katalogów typu `/etc`, `/run`, `/usr`, `/home`
 - Unikaj montowania `/run/docker.sock` do kontenera



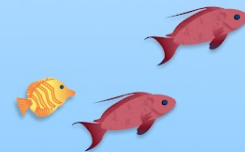
Utwardzanie kontenerów

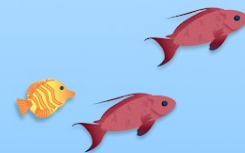


- **Jeżeli korzystasz podman: rozważ użycie --users**
 - Powoduje to skonfigurowanie user namespace
 - Root w kontenerze przestaje być rootem poza nim
 - Losowanie zakresu identyfikatorów użytkowników
 - Wymaga uważnej konfiguracji wolumenów
 - Czyli użycia opcji uidmap i gidmap
- **Jeżeli chcesz pozwolić zwykłym użytkownikom na stawianie kontenerów – użyj podman zamiast docker**
- **Rozważ użycie alternatywnych runtime'ów**
 - Więcej za tydzień



Pytania?





**Dziękuję
za uwagę**

